

TECHNICAL REBUILD TUTORIAL

ClassVault Rebuild Blueprint

How to build the campus study hub again, with a cleaner foundation and a stronger AI layer.

ClassVault project guide

Version 1.0 · 2026.06

Generated with Kami

| Contents

01	Rebuild Thesis	03
02	What To Do First	04
03	Recommended Stack	06
04	Data Model	08
05	Trust, Uploads, And Moderation	10
06	Search, Community, And AI	12
07	Frontend And Delivery Plan	15
08	Quality Bar And Launch	18

| Rebuild Thesis

If I rebuilt ClassVault from scratch, I would not start with screens. I would start with the trust model: who can upload, who can see, who can verify, who can moderate, and what evidence the AI is allowed to use.

ClassVault is not just a notes app. The useful version is a campus knowledge network where study material becomes searchable, discussable, reusable, and AI-readable without becoming messy or unsafe. The current project already proves several important product primitives: notes, moderated uploads, full-text search, saves, ratings, comments, notifications, collections, study rooms, leaderboards, AI roadmaps, exam plans, and campus verification.

The smarter rebuild keeps the same core direction but changes the order. Build the durable spine first: institution, course, note version, file ledger, moderation event, AI usage event, document chunk, citation, and audit log. Then build the interface around those objects. That sequence makes AI features easier, search better, moderation safer, and paid institutional plans less painful later.

What I Would Keep

- Next.js App Router, React, TypeScript, and pnpm. The current stack fits a product with server-rendered pages, route handlers, and client-side interactive islands.
- Postgres with Prisma. The app needs relational integrity, full-text search, moderation history, usage counters, and later vector search. Postgres is the right center.
- Zod at every boundary. AI output, upload metadata, form input, and route bodies all need runtime validation, not just TypeScript.
- S3-compatible object storage. Keep signed upload and download URLs, a storage-key ledger, and file metadata verification.
- Thin route handlers with server modules. The current repo already follows this pattern under `lib/server/` ; preserve it.

What I Would Change

Area	Current shape	Rebuild change
Data model	Strong notes core, institution and course exist but are still emerging.	Make Institution, Course, Syllabus, NoteVersion, DocumentChunk, and AIUsageEvent first-class on day one.
AI	Direct provider module with Gemini primary and OpenAI fallback.	Use a provider abstraction plus AI SDK-style streaming, structured outputs, tool calls, retrieval, and per-institution quotas.
Uploads	File validation, storage keys, direct upload, moderation.	Add async file processing: virus scan, text extraction, OCR fallback, duplicate detection, embeddings, and preview generation.
Search	Postgres full-text search plus filters and trending.	Use hybrid search: full-text for exact course intent, vector search for semantic questions, and citation-backed AI answers.
Product flow	Many features shipped in parallel.	Sequence trust, corpus, search, and moderation before advanced community and AI features.

CORE PRINCIPLE

Build ClassVault as a trusted study corpus first. The AI should be a layer over verified, searchable, permissioned material, not a magic text box sitting beside loose uploads.

| What To Do First

The first week should answer product and data questions, not animation and landing-page questions. A clean first week saves months of rewrites.

Day 1: Write The Product Contract

Create a one-page product contract before opening the editor. It should define the users, promises, non-goals, and trust rules. For ClassVault, the user roles are student, contributor, moderator, admin, and institution owner. The promise is simple: students find useful, campus-relevant material faster, and AI can explain or plan using the material with citations.

Decision	Recommended answer	Why it matters
Primary user	Student preparing for coursework or exams.	Keeps the UI focused on search, saved notes, progress, and exam pressure.
Supply source	Student uploads, seeded official samples, and institution-curated material.	Prevents the library from depending only on cold-start uploads.
Trust rule	Files are not fully trusted until validated and reviewed.	Protects the app from spam, copyrighted dumps, unsafe files, and low-quality notes.
AI rule	Answers must cite notes, pages, chunks, or clearly say when no source exists.	Prevents plausible but unsupported study advice.
Business rule	Institutions pay for governance, analytics, quotas, and verified vaults.	Gives the product a monetizable surface without charging every student first.

Day 2: Define The Non-Negotiable Flows

Map these flows before UI polish: sign up, verify campus, upload resource, moderate resource, search library, save note, ask note, report note, resolve report, and review institution analytics. If a screen does not support one of these flows, it is probably not part of the first build.

```
student signs up
  -> verifies campus email
  -> uploads file and metadata
  -> file enters processing and moderation
  -> approved note appears in course library
  -> classmates search, save, comment, rate, download
  -> AI can cite chunks from approved notes
  -> moderators handle reports and quality issues
```

Day 3: Create The Architecture Map

FIGURE 1 REBUILD ARCHITECTURE MAP

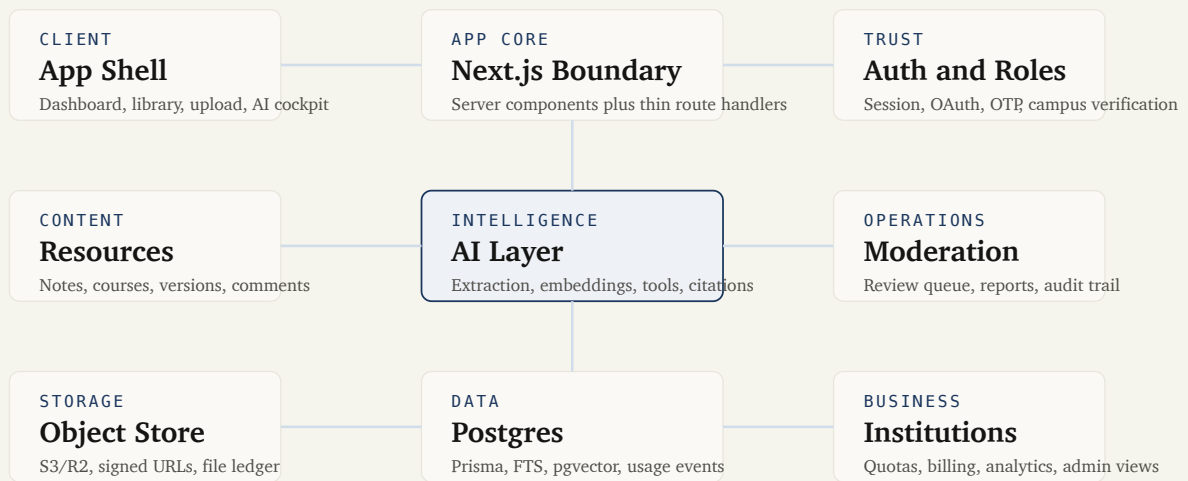


Figure 1. The rebuilt app should make the AI layer depend on trusted content, storage, and Postgres rather than bypassing them.

Days 4-5: Bootstrap The Repo

Use a boring setup. The product is already ambitious; the build system should not be. Start with pnpm, Next.js App Router, TypeScript strict mode, Tailwind, Prisma, Vitest, Playwright, ESLint, and a CI workflow that runs lint, typecheck, tests, Prisma validation, and build.

```
pnpm create next-app classvault --ts --app
cd classvault
pnpm add zod prisma @prisma/client pg @prisma/adapter-pg
pnpm add lucide-react motion @vercel/analytics @vercel/speed-insights
pnpm add -D vitest @playwright/test eslint-config-next tsx
pnpm prisma init
```

Do not add payment, chat, vector search, and background jobs on day one. Put placeholders in the architecture, not in the dependency list. Add the dependency when the first production path needs it.

Recommended Stack

I would mostly keep the current stack. The efficiency gain is not a dramatic framework switch; it is a stricter architecture, a better data model, and an AI pipeline designed before the AI features multiply.

Default Stack

Layer	Choice	Reason
App framework	Next.js App Router	Good fit for server-rendered product pages, route handlers, metadata, dashboard routes, and selective client interactivity.
UI runtime	React 19 with TypeScript strict mode	Matches the existing repo and supports typed, componentized product surfaces.
Styling	Tailwind 4 plus local design tokens	Fast to build, easy to keep dense operational UI consistent, and already aligned with the project.
Database	Postgres on Neon or another managed Postgres	Relational core, full-text search, transactions, analytics, and pgvector path in one system.
ORM	Prisma 7	Strong schema readability, migrations, generated client, and enough escape hatches for raw SQL search queries.
Storage	S3-compatible storage such as R2 or AWS S3	Cheap durable file storage, signed upload/download URLs, CDN path later.
Email	Resend for speed, SES for scale or AWS consolidation	Email OTP, verification, moderation notifications, and institution digests.
AI	Vercel AI SDK-style abstraction plus OpenAI/Gemini providers	Streaming, structured outputs, provider flexibility, tool calls, and consistent UX hooks.
Testing	Vitest, Playwright, and AI eval fixtures	Unit tests for pure logic, e2e for flows, evals for prompt and retrieval quality.
Observability	Vercel Analytics/Speed Insights plus structured logs and error tracking	Product usage, performance, failures, and AI cost need to be visible early.

When I Would Choose A Different Stack

There are only a few cases where I would move away from the current direction. If speed matters more than custom campus logic, Supabase can replace custom auth, storage, and some admin tooling. If the product becomes mobile-first with heavy native features, move the backend into a separate API service and keep Next.js as the web client. If institution analytics becomes the main business, add a warehouse later rather than bending the app database into a BI system.

Scenario	Alternative	Tradeoff
Solo MVP in under 2 weeks	Supabase Auth, Storage, Postgres, Edge Functions	Faster setup, less custom security code, but campus verification and moderation rules become platform-shaped.
Large multi-platform app	Next.js frontend plus separate Fastify/Nest API	Cleaner service boundary for mobile clients, but more deployment and auth complexity.
Institution enterprise from day one	Postgres app DB plus warehouse/event pipeline	Better analytics, but unnecessary until there is real institutional usage volume.

Recommended Repository Shape

```
app/
  api/                route handlers: thin HTTP boundary only
  app/                authenticated product routes
components/
  app-shell/          navigation, command palette, notifications
  views/              route-level client views
  notes/              reusable note cards, previews, controls
lib/
  server/             auth, db, storage, AI, moderation, domain logic
  api-types.ts        serialized response types
prisma/
  schema.prisma       source of truth for data contracts
  migrations/         reviewed database history
tests/
  e2e/                Playwright user flows
  *.test.ts           domain logic and route helper tests
docs/
  architecture/       decisions, runbooks, AI policies
```

STACK CALL

Keep the Next.js plus Prisma plus Postgres direction. Spend the rebuild energy on domain modeling, background processing, retrieval, evals, and moderation tooling.

| Data Model

A better ClassVault starts with a better schema. The database should describe the product you want in year two, not only the upload form you need in week one.

Start With These Core Entities

Entity	Purpose	Build now?
User	Student, moderator, admin, institution owner, verification state, role.	Yes
Institution	College identity, domain, plan, quotas, billing, analytics scope.	Yes
Course	Stable home for notes, PYQs, syllabus, followers, top contributors.	Yes
Syllabus	Units, topics, outcomes, exam weight, prerequisite graph.	Yes, simple version
Note	Published resource record with owner, course, metadata, status, ratings.	Yes
NoteVersion	File replacement history, moderation per version, old storage keys.	Yes
UploadedFile	Storage ledger that prevents fake storage keys or wrong file metadata.	Yes
DocumentChunk	Extracted text, page, section, embedding id, moderation eligibility.	Before AI Q&A
AIUsageEvent	User, institution, feature, model, tokens or cost estimate, status.	Before public AI
AuditLog	Security and moderation-sensitive actions.	Before staff tooling

Schema Pattern

Use Prisma for relational models and migrations. Use raw SQL where Prisma cannot express the exact Postgres feature cleanly, such as ranked full-text search, triggers for search vectors, and vector indexes. This is not a failure of Prisma; it is the right division of labor.

```

model Institution {
  id                String    @id @default(cuid())
  name              String    @unique
  slug              String    @unique
  emailDomain       String?   @unique
  plan              String    @default("free")
  quotaStorageBytes BigInt?
  quotaAiRequestsMonth Int?
  users             User[]
  courses           Course[]
}

model Course {
  id                String    @id @default(cuid())
  institutionId     String?
  code              String
  name              String?
  subject           String?
  notes             Note[]
  syllabusUnits     SyllabusUnit[]

  @@unique([institutionId, code])
}

model Note {
  id                String    @id @default(cuid())
  courseId          String?
  ownerId           String
  title             String
  status            NoteStatus @default(PENDING)
  currentVersionId String?
  versions          NoteVersion[]
  chunks            DocumentChunk[]
}

```

Indexes To Plan Early

- Course lookup: `institutionId`, `code` and `subject`, `semester` .
- Library browsing: `status`, `createdAt` , `courseId`, `status` , and tag join indexes.
- Moderation queue: `status`, `reviewedAt` and reviewer history.
- Trending: `noteId`, `createdAt` on download events.
- Notifications: `userId`, `readAt`, `createdAt` .
- Rate limits: key as primary id with window start and updated time.
- AI retrieval: full-text vector, embedding vector index, `noteId`, `pageNumber` on chunks.

Migration Discipline

Use migrations as product history. Avoid changing a model and immediately coupling it to a large UI change. For risky schema work, ship in three moves: add nullable fields and backfill, read from the new fields with fallback, then enforce constraints after production data proves the migration.

DATA MODEL RULE

Do not make AI features store their own invisible truth. Every AI answer should resolve back to Course, Note, NoteVersion, DocumentChunk, User, and Institution records.

| Trust, Uploads, And Moderation

ClassVault lives or dies by trust. A fast upload flow is not enough; the app needs a defensible path from raw file to approved study resource.

Authentication And Campus Identity

Keep the custom cookie-session approach if campus verification, role bootstrapping, and institution scoping are central. Store only hashed session tokens, use HTTP-only cookies, and keep role checks server-side. Google OAuth and email OTP can coexist, but both should feed the same User and Session tables.

Concern	Implementation rule	Test
Session safety	Hash raw session tokens before database storage.	Unit test token hashing and expiry.
Campus access	Restrict or mark verified users by allowed email domains and verification codes.	E2E sign-up plus verification path.
Roles	Use server-side <code>requireRole</code> checks on every staff route.	Route tests for student denial and moderator access.
Institution scope	Pass institution id into rate limits, queries, and analytics.	Integration tests with two institutions.

Upload Pipeline

The upload pipeline should be explicit. First, validate the intended file name, type, size, and user. Second, create a storage key generated by the server. Third, give the client a signed upload URL. Fourth, verify the uploaded file metadata. Fifth, create a pending note and processing job. Sixth, extract text and preview data. Seventh, send it to moderation.

```
POST /api/uploads/presign
-> validate user, mime type, size
-> create storage key and UploadedFile ledger row
-> return signed PUT URL
```

client uploads file directly to object store

```
POST /api/uploads
-> verify storage key belongs to user
-> verify file size and type
-> create NoteVersion and Note in PENDING state
-> enqueue processing and moderation jobs
```

Moderation System

Build moderation before growth. The current code has the right instinct: staff actions are transaction-wrapped and recorded as moderation events. Expand that into a full review console with reasons, duplicate warnings, text extraction preview, report backlog, and audit log search.

Moderation feature	Why it belongs early	AI assist?
Approve, reject, hide, restore	Basic lifecycle control.	Suggest likely decision, never auto-publish at first.
Report resolution	Required once comments and public notes exist.	Summarize report context and prior actions.
Duplicate detection	Prevents ten copies of the same PYQ file.	Use file hash plus semantic title and chunk similarity.
PII and unsafe content detection	Protects students and institutions.	Flag emails, phone numbers, IDs, answer-key leaks, spam.
Copyright risk tagging	Campus resources can include copyrighted books or paid material.	Flag suspicious source patterns for human review.

Storage Safety

Do not let the client invent arbitrary storage keys. Keep server-generated keys, sanitize file names, track uploaded file rows, and verify the final object. Public URLs are fine only for approved material; otherwise use short-lived signed download URLs.

TRUST RULE

The safest upload flow is not the one with the fewest steps. It is the one where every step can be explained, audited, retried, and revoked.

| Search, Community, And AI

AI should arrive after the corpus exists. Search proves the material is structured; AI then becomes a better way to navigate that structure.

Search In Three Layers

Layer	Use case	Implementation
Filters	Semester, subject, course, tag, file type, owner, saved.	Prisma queries and indexed relational fields.
Full-text search	Exact phrases and multi-word course intent.	Postgres <code>tsvector</code> , <code>websearch_to_tsquery</code> , ranked ids, fallback to substring fields.
Semantic search	Ask for concepts even when words differ.	Embeddings on DocumentChunk records with hybrid ranking.

Community Features Worth Building

Community should reinforce study value, not become generic social noise. Keep comments attached to notes, collections attached to exam sprints or courses, rooms attached to focused sessions, and leaderboards attached to useful contribution, not raw activity.

- Comments and one-level replies: enough for Q&A without turning every note into a forum thread.
- Collections: bundles for exam sprints, unit packs, lab references, and public share links.
- Study rooms: lightweight focus sessions tied to courses or exam windows.
- Leaderboards: count approved contributions, helpful comments, ratings from others, and downloads excluding self-downloads.
- Notifications: moderation decisions, comment replies, collection updates, course follower updates, and digest emails.

AI Feature Roadmap

Priority	Feature	How it should work
P0	Upload metadata assistant	Suggest title, description, tags, course, unit, and PYQ flag from file name plus extracted first pages.
P0	AI roadmap	Use course, syllabus, weak topics, saved notes, and approved resources to generate day-by-day plans.
P0	Exam sprint	Rank high-yield topics, produce a crash plan, and attach matching PYQs or notes.
P1	Ask your notes	Answer questions from approved DocumentChunks only; cite note title, page, and chunk id.
P1	Flashcard and quiz generator	Generate cards from a note section; track confidence and resurface weak items.
P1	Semantic duplicate detector	Compare upload text and embeddings against existing course notes before moderation.
P2	Moderator copilot	Summarize file content, risk flags, duplicate matches, and recommended moderation reason.
P2	Institution insights	Show weak courses, upload gaps, high-demand topics, unanswered questions, and moderation backlog.
P3	Personal study coach	Use saved notes, tasks, room sessions, quizzes, and deadlines to adapt weekly plans.

AI Architecture

Keep prompts out of route handlers. Create feature-specific AI modules with typed inputs, structured outputs, model/provider settings, retries, and test fixtures. Every model response should be parsed through Zod. Every production call should create an AIUsageEvent and include a feature name, user id, institution id when present, provider, model, status, latency, and estimated token or request cost.

```
lib/server/ai/  
  providers.ts           provider selection and retries  
  structured.ts         schema-bound JSON generation  
  retrieval.ts          hybrid search and citation assembly  
  features/  
    roadmap.ts  
    exam-plan.ts  
    note-suggestions.ts  
    ask-notes.ts  
    moderation-assist.ts  
  evals/  
    fixtures.ts         stable inputs and expected properties
```

AI Guardrails

- Require citations for note-grounded answers; no citation, no confident answer.
- Separate creative study planning from factual Q&A. Planning can be flexible; Q&A must be source-bound.
- Rate-limit by user, IP, and institution. Track quota before calling the model.
- Use background jobs for document extraction, embeddings, duplicate detection, and batch summaries.
- Keep an eval set for every AI feature. Test schema validity, citation presence, refusal behavior, and latency budget.

Frontend And Delivery Plan

The frontend should feel like a study operating system: dense, calm, fast, and obvious. Avoid making the signed-in app look like a landing page.

Design Direction

Use a restrained product UI with a strong app shell, predictable navigation, and focused work surfaces. Keep cards for repeated items such as notes, tasks, collections, and queue rows. Use tables for moderation and admin analytics. Use drawers or side panels for note detail, citations, upload progress, and AI conversations.

Surface	Design idea	Important state
Dashboard	Study cockpit: tasks, current sprint, saved notes, trending course resources, room schedule.	Empty account, verified account, exam week, offline or failed fetch.
Library	Faceted browser with search, filters, quick preview, save, rate, download, collection add.	No results, semantic results, pending note hidden, citation jump.
Upload	Wizard: file, metadata, AI suggestions, preview, policy confirmation, moderation status.	Invalid mime, too large, duplicate warning, upload retry.
AI Roadmaps	Inputs on left, generated plan on right, linked resources and progress tracking.	Quota hit, no resources found, regenerate with constraints.
Ask Notes	Chat panel grounded in selected course or collection with visible citations.	No source answer, conflicting notes, citation unavailable.
Review Queue	Dense staff table plus detail pane: file preview, extraction, risk flags, actions.	Already reviewed, duplicate, report escalated.
Institution Admin	Analytics dashboard: uploads, approvals, active courses, AI usage, storage, reports.	Quota warning, plan inactive, domain unverified.

Component Rules

- Use Server Components for data-heavy route shells, and client components for forms, search controls, optimistic saves, drawers, and command palettes.
- Keep note cards stable in size. Do not let ratings, tags, or long course names resize the entire grid.
- Use icons for repeated actions: save, upload, search, filter, report, approve, reject, hide, restore.
- Make keyboard search a first-class workflow. Students should jump from dashboard to any note, course, room, or collection without browsing the sidebar.
- Show source confidence in AI answers. Citations should be visible, clickable, and tied to note previews.

- Design mobile as a real product, not a collapsed desktop table. Library search, upload status, and AI roadmaps must work on phone screens.

Delivery Plan

Phase	Build	Exit criteria
0	Product contract, schema draft, repo bootstrap, CI, env examples.	Local dev, migration, seed, lint, typecheck, and smoke test pass.
1	Auth, user profile, institution, campus verification, roles.	Student, moderator, admin paths tested.
2	Upload, storage ledger, note creation, moderation queue.	Pending upload can be approved or rejected; invalid files blocked.
3	Library, filters, full-text search, note preview, save, rate, download.	Students can find, save, and download approved material.
4	Comments, reports, notifications, collections.	Community actions generate correct notifications and moderation paths.
5	AI metadata, roadmap, exam plan with schema validation and quotas.	AI calls are rate-limited, logged, typed, and useful without retrieval.
6	Text extraction, chunks, embeddings, ask notes, citations.	Answers cite approved notes and refuse unsupported claims.
7	Institution analytics, billing, admin reports, launch runbooks.	Pilot college can manage users, quotas, content, and reports.

First 30 Days

Week 1: product contract, repo, schema, auth, institution and course seed data. Week 2: upload, storage, moderation, library basics. Week 3: search, saves, ratings, downloads, dashboard, notifications. Week 4: AI metadata, roadmap, exam plan, admin review polish, tests, pilot checklist.

Do not wait until the end to add tests. The current repo already benefits from unit tests for pure logic and Playwright specs for core flows. Preserve that habit from the first feature.

| Quality Bar And Launch

A smarter rebuild is not finished when the features exist. It is finished when unsafe paths fail closed, AI behavior is measured, and a pilot college can trust the workflow.

Test Matrix

Area	Tests	Examples
Domain logic	Vitest unit tests	rating aggregation, notification formatting, comment threading, rate-limit keys.
Server routes	Integration tests with mocked auth/storage/AI	upload creation, moderation action, report resolution, AI schema failure.
Browser flows	Playwright	sign in, upload, approve/reject, library search, collections, comments, trending.
AI features	Fixture-based evals	valid JSON, citations present, unsupported answer refusal, no PII leakage.
Database	Prisma validate, migrations, seed smoke	schema compiles, migrations replay, seed creates demo courses and notes.
Release	Build and health checks	Next build, deep health route, storage access, email provider check.

Security Checklist

- HTTP-only session cookie, token hashing, expiry, and sign-out cleanup.
- Server-side role checks for every moderator, admin, and institution route.
- Rate limits for auth, uploads, comments, downloads, reports, and every AI endpoint.
- Storage keys generated only by the server; path traversal rejected.
- Mime type, size, and ledger verification before creating public note records.
- Report and moderation events retained for audit history.
- AI prompts avoid secrets and raw unnecessary personal data.
- Public collection and note routes enforce visibility rules.

Launch Sequence

Launch with one pilot college or one department, not a vague global campus market. Seed 30-50 high-quality notes across 5-8 courses, recruit 5-10 contributors, and appoint at least one moderator. Make the first pilot about weekly retention and upload quality, not vanity signup counts.

Milestone	Target	Decision
Seed quality	At least 5 useful resources per pilot course.	Do students find something useful before uploading?
Search success	Most common course queries return relevant results.	Do filters, FTS, and course pages work?
Upload health	Moderation can clear pending notes within 24 hours.	Is the review workflow manageable?
AI usefulness	Roadmap/exam outputs are accepted or saved by users.	Does AI change study behavior?
Institution value	Admin can see useful upload, usage, and risk metrics.	Would a college pay for governance?

The Final Shape

The better ClassVault is not simply faster or more AI-heavy. It is more legible. Students understand where answers came from. Moderators understand what changed. Institutions understand usage and risk. Developers understand where business logic lives. That clarity is what makes the app easier to scale.

MONDAY MORNING START

Write the product contract, draw the schema, seed the first courses, and build the upload-to-moderation path before anything else. Once trusted notes exist, search, community, and AI all become easier to build well.

| Appendix

Sources Checked

- Local ClassVault source: README.md , docs/roadmap.md , package.json , prisma/schema.prisma , and targeted files under lib/server/ and components/ .
- Current docs lookup through Context7 for Next.js App Router, Prisma, and Vercel AI SDK.
- Official reference locations: https://nextjs.org/docs , https://www.prisma.io/docs , and https://ai-sdk.dev/docs .

Build Commands

```
pnpm install
cp .env.example .env
pnpm prisma validate
pnpm db:migrate
pnpm db:seed
pnpm lint
pnpm typecheck
pnpm test
pnpm test:e2e
pnpm build
```

Definition Of Done

Feature	Done means
Upload	Valid files upload, invalid files fail, moderation can approve/reject, storage access is verified.
Library	Search, filters, note preview, saves, ratings, downloads, and empty states work on mobile and desktop.
AI roadmap	Output schema is validated, rate-limited, logged, and useful without hallucinated resources.
Ask notes	Every factual answer includes citations or refuses the question.
Moderation	Every staff action has a reason, actor, timestamp, and reversible status where appropriate.
Institution plan	Users, quotas, domains, analytics, billing state, and admin views share the same institution id.

Glossary

Trusted corpus - The approved, permissioned set of notes and chunks that search and AI are allowed to use.

DocumentChunk - A piece of extracted note text with page and section metadata, used for semantic search and citations.

Hybrid search - Ranking that combines exact full-text matches, relational filters, and vector similarity.

AIUsageEvent - A durable record of each model call for quota, cost, debugging, and abuse analysis.